



Python for Fixed Income

COURSE 7 · NLP AND LLMS

Bag of Words and TF-IDF

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From tokens to numbers

1. Count

Bag of words: one row per document, one column per token

2. Weight

TF-IDF: each count times how rare the word is

3. Compare

Cosine similarity: one number for two documents

```
textkit.term_counts(docs)  
textkit.tfidf_by_hand(counts)
```

```
textkit.idf(doc_freq, n_docs)  
textkit.cosine_similarity_matrix(A, B)
```

Each one tested against scikit-learn to 1e-12, on 226 statements and Article 4.

Real documents, described

An FOMC statement's output

Allowed names

count of words from the course's list

dictionary score

the action the statement states

Never called

hawkish or dovish

a signal or a policy view

a forecast of rates

An indenture's terms

Allowed names

the indenture states

extracted value, with its quote

passed the source check

Never called

loose, tight, weak, strong

covenant-lite or protective

a credit view or recommendation

Bag of words

226

rows: one per statement

1,686

columns: one per distinct token

0.0983

share of the table not 0

The raw text gives four other columns

Only after `normalize_text`: and-in, banks-the (two em dashes made hyphens). Only on the raw text: s, survey (the curly apostrophe and U+2011 split a word).

212 vote paragraphs left out first: no person's name becomes a column.

Document frequency and IDF

```
idf = ln((1 + n) / (1 + df)) + 1          # scikit-learn's smooth_idf, the default
```

Term	Statements	IDF
the	226	1.0000
committee	225	1.0044
inflation	205	1.0971
pandemic	19	3.4292
mortgage-backed	108	1.7336

The two ends

In every statement (5 tokens): weight 1.0, never 0. In one statement only (404 tokens): the largest weight.

A word in every document cannot tell one document from another.

A paper on term specificity

January 1972

Karen Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval", Journal of Documentation, volume 28, issue 1, pages 11 to 21.

The course takes the record's facts only. The paper itself was not read for this course.

Source, read 8 October 2026: the Crossref record, api.crossref.org/works/10.1108/eb026526 (DOI 10.1108/eb026526)

TF-IDF by hand: the first statement three ways

By count	
the	19
of	8
in	7
federal	5
rate	5

By TF-IDF	
the	0.4506
of	0.1906
discount	0.1832
in	0.1675
reserve	0.149

With sublinear_tf	
pronounced	0.1686
discount	0.159
record	0.1567
5-3	0.1482
borrow	0.1482

"the" 19 times still leads plain TF-IDF. With $1 + \ln(\text{tf})$, "pronounced" leads.

Counts times IDF, each row scaled to length 1: equal to TfidfVectorizer within 1e-12.

One setting at a time

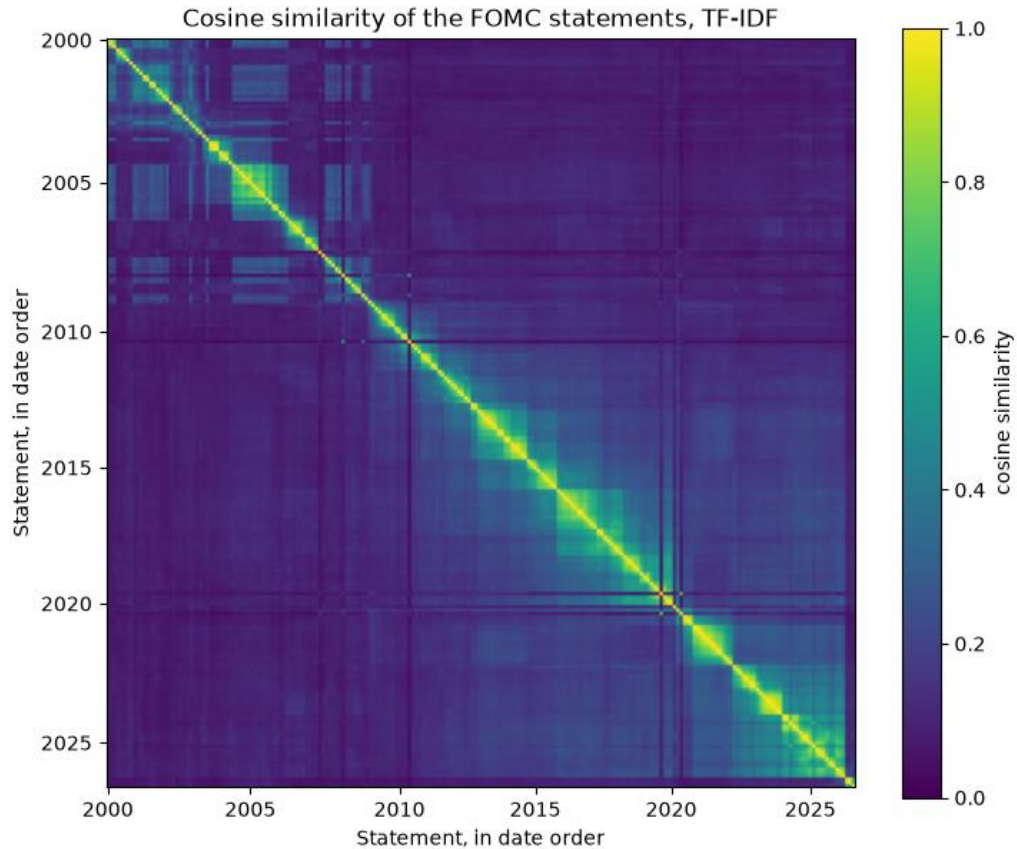
Setting	Columns
scikit-learn's defaults	1,614
the course's token pattern	1,686
+ sublinear_tf=True	1,686
+ min_df=2	1,282
+ ngram_range=(1, 2)	6,143

What each one changes

Default pattern: splits at hyphens, drops one-character tokens.
Sublinear: weights only. min_df=2: drops terms in one statement. Bigrams: pairs of neighbours.

The last row is the course's setting from now on.

Cosine similarity



With the course's setting

Most alike: 2024-01-31, 2024-03-20 0.9699

Least alike: 2004-03-16, 2010-05-09 0.0241

Median with the next statement: 0.7731

Median of all pairs: 0.1129

Least like the rest: 3 unscheduled releases

A cosine measures shared words. It is not a probability.

Distinctive terms by year

Year	The three most distinctive terms
2008	to growth, housing contraction, labor markets
2013	asset, purchases, a highly
2014	asset, asset purchase, purchase program
2021	by at, percent for, on vaccinations
2022	war, demand imbalances, imbalances related
2026	at 3-1, 3-3, 3-3 4

How it is computed

Mean TF-IDF in the year's statements minus the mean in all others; the three largest, ties by term.

Words the statements wrote in those years, nothing more. 3-3 is 3-3/4 cut at the slash.

Article 4 as twenty documents

9 of 20

sections that hold "indebtedness"

1.7419

its idf (the largest possible: 3.3514)

67

times in 4.12, the most of any section

Section	and	Cosine
4.10	4.11	0.5688
4.12	4.15	0.5330
4.12	4.16	0.4663

What 4.10 and 4.11 share most

offer, notes or, or portions, portions: each section sets out an offer to the holders.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

What a bag of words loses

```
Leverage is less than 3.0 and coverage is not less than 2.0.  
Leverage is not less than 3.0 and coverage is less than 2.0.
```

1.0000

cosine as bags of words

1.0000

with bigrams too

0.9512

with trigrams

Same words, opposite tests. A bag of words indexes what a text is about, not what it says.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIF(A2:A227,"*inflation*")`

`=COUNTIF(A2:A227,REPT("a",256))`

`=LN((1+226)/(1+J2))+1`

`=I2*K2`

`=L2/SQRT(SUMSQ(L$2:L$124))`

`=SUMPRODUCT(P1:P6143,Q1:Q6143)/(SQRT(SUMSQ(P1:P6143))*SQRT(SUMSQ(Q1:Q6143)))`

205, as Python

#VALUE!; 255: 0

within 1e-12 (123)

TF times IDF

within 1e-12

0.9699

No tokenizer and no TF-IDF in Excel: once the counts exist, it checks every step after them.

Save the day: read the diff

1. Morning

Commit textkit as day 1 left it

2. Lesson

Four functions, four appends

3. Diff

2 files, 150 lines added, none removed

4. Commit

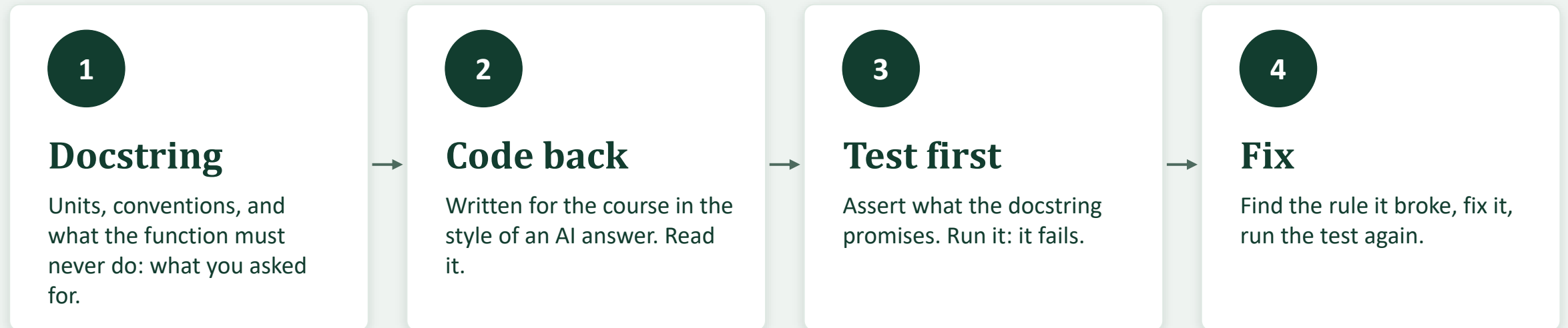
Today's two files

```
8eaae35 Add term_counts, idf, tfidf_by_hand, cosine_similarity_matrix, 14 tests
e03f571 Start of day 2: textkit as day 1 left it, 7 tests
```

pytest: 14 passed. Fragments only append: a minus line is the first thing to read.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: IDF as $\log_{10}(n / df)$: 5 terms weigh 0, and the top term moves from "the" to "pronounced".

Code that runs is not the same as code that is right.

What you can do now

- Count tokens into a bag of words, by hand and with CountVectorizer
- Compute scikit-learn's smooth IDF and TF-IDF by hand, to $1e-12$
- Measure how alike two documents are with a cosine
- Find what sets a year, or a covenant section, apart
- Do every step after the counts in Excel

Week 1, Day 3 Classifying 8-K items with TF-IDF